Contents

Chapter 1

Introduction

1.1 Motivation

A Crazyflie 2.0 measures 9 cm rotor-to-rotor, and when the interaction between vehicles is left unmodelled, planners have adopted conservative separations, Shi et al. [15] cite 60 cm from earlier planning work as one such example. The margin is not arbitrary caution. Below it, the downwash of one vehicle acting on another is strong enough to destabilise the lower one, and the standard remedy is to keep the vehicles far enough apart that the effect can be ignored.

That margin is expensive. It sets the density at which a team can operate, and therefore the size of the space a given number of vehicles requires. Applications that motivate multirotor teams in the first place, inspection in confined spaces, cooperative transport, aerial docking, are precisely those in which the space is not available. In some of them, close approach is the task rather than a hazard to be avoided.

Removing the margin requires a controller that accounts for the interaction rather than avoiding it, and that is difficult for a specific reason. The interaction is not a small perturbation around a known model: it is strongly nonlinear in the relative configuration of the team, growing sharply as vehicles close and decaying quickly with lateral offset, and it is markedly asymmetric, since the vehicle above is affected far less than the one below. It is also hard to derive from first principles in the regime that matters. Simplified models exist for part of the problem: the mean flow from a quadrotor's propellers in hover is well approximated by a turbulent jet beyond about two and a half motor-to-motor distances below it [2], but formation flight operates closer than that, in a near field for which those authors report no closed-form model.

The field's response has been to obtain the interaction from data rather than from theory, and it has worked. Reported results include a reduction in worst-case height error by a factor of two or better, up to four [15], sixteen vehicles flying at a minimum vertical separation of 24 cm [14], and average reductions of 56 % in vertical and 36 % in three-dimensional tracking error for a pair of vehicles [17]. The margin has been substantially reduced.

1.2 Problem

A controller can obtain information about the interaction force in exactly two ways, and the distinction organises the entire field.

It can **measure** the residual online, by comparing the acceleration the vehicle actually experienced against the acceleration its nominal model predicts for the actuation it applied.

This requires no prior knowledge of the interaction and works equally against disturbances never seen before. But the estimate exists only after the disturbance has begun to act, and in its established form it depends on rotor-speed sensing.

Or it can **predict** the residual before it acts, from the relative states of its neighbours, using a model learned offline. This can compensate ahead of the disturbance rather than after it, but the model is only valid over the configurations its training data covered, and outside them its behaviour is not merely degraded but unconstrained.

These are complementary rather than competing: one is reactive and general, the other predictive and specific. A hybrid is therefore an obvious construction, and has been built, for a single vehicle, where its advantage over the reactive method alone was described as small without a payload, and as leaving performance similar to the reactive method with one [4].

The practical question a system designer faces is which to use, and it has no answer in the literature. Not because the individual results are weak, and not because the field avoids systematic comparison, recent work benchmarks online-learning estimators on a single vehicle [6] and fractional-order and classical regulators for close-proximity swarms [1]. It is that the results for *these* families cannot be composed: each is reported against its own baseline, on its own airframe, at its own separations, on its own trajectories, under its own metrics.

1.3 Gap

Representative methods exist for reactive compensation, for predictive compensation, and for their hybrid, and each is individually validated, but, to the best of our knowledge, they have not been evaluated against one another under identical conditions, and the field's reporting conventions make such a comparison impossible to assemble after the fact. A 36 % improvement measured on two vehicles in close proximity and a 31 % improvement measured in a tight formation are not the same quantity; neither can be subtracted from the other, and their ordering says nothing about which method would prevail if both were flown under the same conditions.

This is not a criticism of any individual work. Each is internally valid, and none violated a convention of the field. It is a property of the literature taken as a whole: the results were not designed to compose, and establishing a ranking requires an experiment built for that purpose.

Two narrower gaps follow. The hybrid has been demonstrated only where the residual is *self-induced*, a single vehicle, with effects such as blade flapping, drag and ground forces, optionally carrying a slung payload, and not where it is produced by other vehicles and is therefore a function of a measurable relative state, which is where the predictive component has the most to contribute. And the summation that predictive architectures use to aggregate per-neighbour contributions is known to be an approximation [15, 5], yet the size of the penalty it carries for a deployed model is not reported.

1.4 Research questions

RQ1 Under identical hardware, trajectories, formations and separations, how do reactive, predictive and hybrid interaction-force compensation strategies compare in trajectory-tracking accuracy and residual-force rejection?

RQ2 Does the relative advantage of reactive versus predictive compensation depend on

the interaction regime, separation, relative velocity, and whether the disturbance is quasi-static or transient?

RQ3 How much prediction accuracy does a deep-sets residual model trained only on two-vehicle data lose when applied to three-vehicle formations, and is that loss small enough for the summed per-neighbour architecture to remain the right engineering choice? *This is a measurement of the transfer penalty, not a test of whether superposition holds: it does not, and that is already established* [15, 5].

RQ4 What are the sensing, computational and data requirements of each strategy, and is the added complexity of the hybrid justified in the multi-robot case?

RQ1 is the primary question and the one the experimental design serves. RQ2 carries the most engineering value, because it converts a ranking into a rule about when to use which. RQ3 is deliberately narrower than asking whether interactions superpose, they do not, and that is already reported, and asks instead for the magnitude of the resulting error on a deployed model. RQ4 exists because a method that wins by a small margin while requiring additional sensing, an offline training campaign and a larger compute budget may still be the wrong choice.

The questions are stated with falsifiable hypotheses in Chapter 2 and Chapter 5; each is answerable by the experiments described in Chapter 5, and a negative answer to any of them is reported as a result rather than as a failure.

1.5 Contributions

This thesis sets out to establish the following. Nothing here is a claim about results; the experiments are described in Chapter 5 and reported in the two- and multi-robot results chapters.

1. **A controlled comparison.** To the best of our knowledge the first evaluation of reactive, predictive and hybrid compensation on the same airframe, the same trajectories, the same formations and the same frozen gains, with a single definition of the residual and a measured noise floor. The methodological content is in the word *controlled*. For Strategies 1, 2, 3 and 5 it is unusually strong: those four are not four controllers but four settings of one, differing only in which residual term is active, so a difference between them cannot be attributed to anything else. *This does not extend to Strategies 4 and 6*, which are structurally different controllers — a feedback-linearising law and a receding-horizon optimiser — and for which "identical conditions" means identical trajectories, formations and hardware but not identical structure. The design provides: a frozen scenario library so every strategy meets identical excitation; gains fixed before data collection so a difference cannot be a tuning artefact; one residual definition logged identically under every controller, including those that do not use it; and null conditions in which no interaction is expected, so that a positive result is separable from noise.

2. **The first multi-robot evaluation of the reactive–predictive hybrid,** extending a result established for a single vehicle into the regime where its premise is strongest.

3. **A quantification of what the summed per-neighbour approximation costs** when a model trained on two vehicles meets three. This measures a penalty whose

existence is already established in the literature [15, 5]; the contribution is its magnitude for a deployed model, not the finding that it exists.

4. **An engineering account of cost**: sensing prerequisites, onboard computation and training-data requirements, reported alongside performance so that the comparison supports a decision rather than only a ranking.

Supporting these, but explicitly *not* a scientific contribution, is the apparatus that makes them feasible: the frozen scenario library, the residual measurement path, an onboard network with a weight-upload protocol, an offline training pipeline verified against the compiled controller, and a simulation environment running the same controller source that flies. It is described in Chapter 5 and offered for reuse. A thesis whose contribution is its tooling has not answered a question, and it is listed here only so that the engineering effort is visible.

What this thesis does not claim. No new control method is proposed: every strategy follows published work, and the contribution is the comparison. The residual model is not offered as an improvement on published architectures. Results concern a single airframe and a single class of disturbance; generalisation beyond either is discussed rather than demonstrated. No claim rests on simulation, which is used to disprove implementations and never to validate a method. And no strategy is claimed to be best in general, the useful outcome is conditional, which is RQ2.

A negative result would also be a contribution. If the strategies do not separate, the conclusion is that the choice of compensation family matters less than whether compensation is present at all, and the engineering question becomes one of cost. The experimental design is built so that this outcome is distinguishable from a failed experiment, which is what the null conditions and the frozen gains are for.

Provenance, not pending work. "To the best of our knowledge" is supported by two audits: an arXiv keyword sweep and a cross-database sweep including a citation-graph scan. Both found adjacent comparisons, cited in §2.8, but none spanning these families. Neither audit is exhaustive.

1.6 Scope

The scope is deliberately narrow, because the contribution is a controlled comparison and every degree of freedom left open is an alternative explanation for an observed difference.

Table 1.1: Scope of this thesis.

In scope	Out of scope
Inter-vehicle aerodynamic interaction, principally downwash	Wind, gusts, and other external disturbances
Teams of two and three vehicles	Large swarms
Homogeneous vehicles	Heterogeneous teams
Tracking of pre-planned, collision-free formations	Interaction-aware planning; collision avoidance
Onboard inference, offline training	Online or on-board learning
The force residual	The moment residual, which is measured but not separately compensated

1.7 Outline

Chapter 2 surveys the field as an argument rather than a catalogue: what the disturbance is, the two ways of obtaining information about it, why the injection point into the control law matters, and why the individual results do not compose into a comparison.

Chapter 3 defines the residual this stack measures and logs. Strategies that consume a force residual use that number; Strategy 7 does not form it. It gives both routes to it in one notation, and states the assumptions the formulation makes together with where each is expected to fail.

Chapter 4 describes Strategies 1–6 plus 7a and 7b (eight controllers; seven families, with residual reinforcement learning split by inner loop), and the point at which each injects its knowledge of the interaction, separating in each case what is implemented here from the published method it follows.

Chapter 5 describes the platform, the frozen scenario library, the measurement chain and the protocol, and states the threats to validity the design is built against, including those it does not eliminate.

The two- and multi-robot results chapters report the two- and three-vehicle results. **The discussion chapter** interprets them against the research questions and the stated assumptions, and **the concluding chapter** concludes.

Chapter 2

Background and Related Work

Multirotors flying in close proximity disturb one another aerodynamically. The disturbance is large enough to destabilise a vehicle, and conventional practice avoids it by keeping the vehicles far apart: a safety distance of roughly 60 cm has been used for a quadrotor only 9 cm across [15]. Removing that margin is what makes dense formation flight possible, and doing so requires a controller that accounts for the interaction rather than avoiding it.

This chapter surveys how that has been attempted. It is organised as an argument rather than a catalogue. Section 2.2 establishes what the disturbance is; 2.3 and 2.4 describe the two sources of information a controller can have about it, namely online measurement and offline prediction; 2.5 shows that a prediction is only useful once it enters a control law, and that how it enters differs by controller family; 2.6 covers the alternative of learning the corrective action directly; 2.7 covers combining measurement and prediction. Section 2.8 then examines how these methods are evaluated, and argues that the field's reporting conventions make the individual results impossible to compose into a comparison. That observation is the gap this thesis addresses.

2.1 Multirotor dynamics and baseline control

Each vehicle i in a team is modelled as a rigid body,

$$\dot{p}_i = v_i, \qquad m\,\dot{v}_i = -mge_3 + f_i R_i e_3 + f_{\mathrm{res},i}, \tag{2.1}$$

$$\dot{R}_i = R_i \hat{\omega}_i, \qquad J\dot{\omega}_i = -\omega_i \times J\omega_i + \tau_i + \tau_{\mathrm{res},i}, \tag{2.2}$$

in a world frame with e_3 pointing up, where f_i and τ_i are the collective thrust and body torque produced by the rotors. The terms $f_{\mathrm{res},i}$ and $\tau_{\mathrm{res},i}$ collect every force and moment the nominal model omits. This chapter is concerned with the case in which they are dominated by the aerodynamic interaction with neighbouring vehicles. The formulation is developed in Chapter 3.

The baseline against which interaction-aware methods are measured is a geometric tracking controller on $SE(3)$, which tracks a position trajectory and a single heading direction coordinate-free, avoiding the representation singularities of a local attitude parameterisation [11]. Its guarantees are regional rather than global: exponential stability for initial attitude errors below 90 °, and almost-global exponential attractiveness below 180 °. It is the controller three of the strategies compared in this thesis build upon, and the one used uncompensated as a reference condition. The attitude error function is that work's, verified equation-by-equation against its companion arXiv paper [10]; the torque

law implemented here is a *reduced* form of it, and Chapter 4 states exactly which terms differ.

A second property of the quadrotor is used throughout: differential flatness. Position and yaw, together with their derivatives, determine the full state and the inputs, which allows a feedforward term to be computed directly from a planned trajectory. Tal and Karaman [18] exploit this to track position and yaw derivatives up to fourth order, using flatness-derived feedforward for angular rate and angular acceleration in order to follow jerk and snap. Flatness recurs in Section 2.5 as a structure that a learned correction can either preserve or destroy.

2.2 Aerodynamic interaction in multirotor teams

The dominant interaction in a vertically stacked formation is the downwash of the upper vehicle acting on the lower one. Two properties make it difficult to handle. It is strongly nonlinear, growing sharply as the vehicles close and decaying quickly with lateral offset; and it is asymmetric, since the vehicle above is affected far less than the one below.

Recent experimental work characterises the effect directly rather than inferring it from tracking error. Kiran et al. [9] measure forces *and* torques with load cells on mounted Crazyflies whose roll and yaw are locked and whose motors are driven at a constant hover voltage, and use particle image velocimetry in a centre-plane light sheet to resolve the wake, for a single quadrotor and for a pair; the dynamic cases traverse one vehicle on a rail rather than flying it. Gielis et al. [5] measure six-degree-of-freedom exogenous forces on a multirotor held on a purpose-built load stand at hover thrust, while up to three further vehicles fly freely around it, and report strong nonlinearities in how those interactions aggregate. The vehicle experiencing the disturbance is therefore constrained rather than in free flight. Both measure the disturbance as a transducer load rather than inferring it from a flight controller's residual, actuation is still present in each, at hover thrust, which makes them, *in our reading*, the natural ground truth against which a learned model should be checked. Neither paper makes that claim, and neither validates a learned model of ours or anyone else's.

Simplified physical models exist. Bauersfeld et al. [2] analyse roughly 16 h of hover data across six platforms spanning 0.23–6.3 kg, and show that once the individual propeller wakes have merged the mean flow is well approximated by a turbulent jet — beyond about 2.5 motor-to-motor distances l below the vehicle, a length scale the abstract renders as "drone-diameters". That gives a computationally light alternative to computational fluid dynamics. What it estimates is the *time-averaged magnitude* of the induced flow *in hover*, not an instantaneous six-degree-of-freedom wrench on a follower. For the near field the authors report no closed-form first-principles model and point to computational fluid dynamics instead. Formation flight operates in exactly that near field — our observation, not theirs — which is the practical argument for learning the residual from data rather than deriving it.

2.3 Reactive compensation: measuring the residual online

The first family estimates the residual from its effect and requires no prior model of the interaction. Incremental nonlinear dynamic inversion (INDI) compares measured acceleration with the acceleration the nominal model predicts for the actuation actually applied, and attributes the difference to unmodelled forces, which it then cancels incrementally.

Smeur et al. [16] cascade an inner angular-acceleration INDI loop with an outer linear-acceleration INDI loop on a single Parrot Bebop running Paparazzi at 512 Hz; the position and velocity loops feeding the acceleration reference remain proportional-derivative rather than a third incremental stage. They demonstrate disturbance rejection indoors by flying in and out of a 10 m/s open-jet exhaust every 14 s. In two *separate* experiments they show the method does not depend on frequent position updates: indoors with motion capture deliberately downsampled to 4 Hz, and outdoors taking off and holding position on a consumer GNSS module in roughly 5.1 m/s wind. Tal and Karaman [18] use INDI for robust tracking of linear and angular accelerations in aggressive flight on a single custom 609 g vehicle, and state the property that defines the family: no parametric model of the aerodynamic effects is required. That is narrower than it sounds, mass, inertia, motor time constant, control effectiveness and the throttle map are all still identified; it is the *drag* model that is dispensed with.

The same work also exposes the family's cost. Tracking snap requires direct control of body torque, which those authors achieve using closed-loop motor speed control based on optical encoders on their own vehicle's motors [18]; encoders are what *that* snap-and-torque path required, not a general prerequisite for INDI. Reconstructing the applied wrench well enough to difference it against measurement does, however, generally require some form of rotor-speed feedback, Smeur et al. likewise use motor-speed measurement [16], which is additional hardware or firmware support that must be fitted, calibrated and maintained on every vehicle. A second limitation is structural rather than practical: the estimate is delayed by the sensing and filtering chain, and it exists only once the disturbance has already perturbed the vehicle. A reactive controller cannot, even in principle, act before the disturbance arrives.

That the sensing requirement is not fundamental has since been shown. Cobo-Briesewitz et al. [4] demonstrate that a neural network can generate smooth approximations of INDI's residual estimate without rotor-speed measurement, which is discussed in Section 2.7.

2.4 Predictive compensation: learning the residual from relative state

The second family predicts the residual before it acts, from the relative states of the neighbours, using a model learned offline from flight data. Because the interaction is a function of the team's relative configuration, a model with the right inputs can act ahead of the disturbance rather than after it.

Shi et al. [15] combine a nominal dynamics model with a regularised permutation-invariant deep neural network that learns multi-vehicle interaction, and use the learned model as a feed-forward term in a decentralised nonlinear tracking position controller; geometric $SE(3)$ control appears only as an optional attitude inner loop, and the learned force itself is the vertical component $f_{a,z}$, evaluated onboard the STM32. Training separate networks on data from two, three and four vehicles respectively, they report worst-case height error reduced by a factor of two or better, and up to four times smaller in the most favourable case. In a two-vehicle swapping task the baseline reaches 9.4 cm; the network trained on *four* vehicles brings this to 2.4 cm, while the network trained on two vehicles reaches 2.7 cm. The swap experiments use vertical neighbour spacings of 0.20–0.25 m, with 25 cm stated explicitly for the three-vehicle case. Generalisation is directional rather than universal: networks trained on three or four vehicles improved five-vehicle flight, whereas a network trained only on two vehicles did *not* generalise, performing worse than the baseline on three- and four-vehicle tests.

Neural-Swarm2 [14] extends this to heterogeneous teams using heterogeneous deep sets, whose permutation-invariant architecture accepts any number of neighbours. That is a structural property rather than an accuracy claim: the networks are trained on at most three vehicles (at most two neighbours), and the paper describes prediction at sixteen robots as relatively less accurate. Two design choices are worth noting. Spectral normalisation is applied, which the abstract describes as yielding stability and generalisation guarantees; in the body this rests on Lipschitz bounds together with empirical generalisation and a bounded-residual assumption used in the proofs, rather than a formal guarantee on unseen data. The learned residual is used in interaction-aware motion planning as well as in tracking control. They report flying a heterogeneous team of sixteen robots at a minimum vertical distance of 24 cm, and, separately, up to a threefold reduction in worst-case tracking error against a baseline nonlinear tracking controller with the interaction force set to zero. The two figures are reported in the abstract and we do not assume they describe the same experiment: on the sixteen-robot flight itself the body reports the maximum z-error of a small vehicle falling from 15 cm to 10 cm, a factor of about 1.5.

Subsequent work has focused on making these models cheaper to obtain rather than larger. Smith et al. [17] exploit the rotational symmetry of the problem with an $SO(2)$-equivariant model, and report that five minutes of real-world flight data attains a lower validation error than learning baselines trained on more than fifteen — a data-efficiency result on prediction accuracy rather than a closed-loop tracking one. Deployed online within an optimal feedback (LQR) controller on two identical vehicles, the model reduces three-dimensional tracking error by 36 % and vertical tracking error by 56 % on average.[1] On a lateral transect flown beneath the hovering vehicle they report mean three-dimensional position error falling from 0.154 m to 0.098 m, close to 36 %, with vertical improvements of 55 % and 49 %. The platform is not a Crazyflie but a custom quadrotor of roughly 0.26 m span and 0.7 kg, tracked by OptiTrack and controlled from a Raspberry Pi at 50 Hz with a 45 Hz network update. The closest *training* stage is about 0.5 m — some two body-lengths — while the evaluation manoeuvres are flown at 0.6–0.8 m, together with a lemniscate flown beneath the hovering vehicle at $z = -2.5\,m$. The authors attribute the efficiency to exploiting the problem's latent geometry rather than learning it from data.

The superposition assumption The deep-sets architectures above aggregate per-neighbour terms by summation, which commits them to a form of superposition. The literature is explicit that this is an approximation. Shi et al. [15] observe that with more than two vehicles the aerodynamic effect is not a simple superposition of each pair. Gielis et al. [5] make the same point as their motivation, an accurate model of the downwash from one vehicle is unlikely to suffice for predicting the aggregate from several. Their abstract frames this as modelling the formation as a graph; the implementation learns the aggregation function with a sum-pooling deep-sets predictor, contrasted against a linear-summation baseline. Generalisation to a fourth vehicle is claimed at abstract level, while the reported tables stop at three neighbours and the conclusion is pessimistic about extrapolating further, so we do not treat it as a demonstrated held-out result.

Their measurements refine the picture rather than simply refuting summation. They report that interactions fall into three regimes depending on the formation and its location in the state space: a chaotic, noise-dominated regime; a linear regime *well estimated by*

[1] The paper is internally inconsistent on whether the 36 % figure is three-dimensional or lateral: we follow the abstract and the translation-result body, which give three-dimensional and vertical. Its contributions list instead calls the 36 % figure lateral.

linear summation; and a nonlinear regime requiring a nonlinear model. Superposition is therefore not uniformly wrong but regime-dependent, which makes the practical question one of how much accuracy is lost, and where. This is their own load-stand result, obtained independently of Shi et al.'s free-flight observation cited above.

Summed architectures nevertheless remain in wide use, because a controller does not require an exact model, only one whose remaining residual is smaller than the one it removes. What is not reported is the size of the penalty that the simplification carries for a deployed model, which is the question this thesis takes up in the multi-robot results chapter.

2.5 Injecting the prediction: the controller matters

A predicted residual is inert until it enters a control law, and the manner of entry differs by controller family. Treating "predictive compensation" as a single method would obscure differences that are as large as those between the families themselves.

Feedforward into a geometric controller. The simplest injection adds the predicted force to the desired force of the baseline controller, leaving its structure unchanged. This is the form used with the permutation-invariant models of Section 2.4, whose baseline is a nonlinear tracking controller [15, 14].

Feedback linearisation with a flatness-preserving residual. A learned correction added to a nominal model can destroy differential flatness, and with it the applicability of flatness-based planning and control. Yang et al. [19] show that residuals with a lower-triangular structure preserve both the local flatness of the system and its original flat output for systems whose nominal model admits a pure-feedback form, and give a constructive procedure for recovering the diffeomorphism of the augmented system. The residual there is a *structured correction inside the state derivative* rather than a free six-degree-of-freedom wrench — in their worked example it appears only in the velocity equation. Their validation is in simulation on a planar, two-dimensional quadrotor; there is no hardware, no three-dimensional vehicle and no multi-robot downwash experiment, and they list a physical quadrotor as future work. Hsieh et al. [8] apply the idea to tight formation flight, learning a physics-informed residual that keeps the joint multi-quadrotor system differentially flat and using the preserved structure to design a feedback-linearising controller that cancels the disturbance by feedforward. Their abstract reports a 31 % reduction in average tracking error against nominal baselines; the body does not restate that aggregate, reporting instead, on hardware, 24 % RMSE and 30 % maximum-z improvement for ROM-FBL over nominal FBL, and a further 29 % RMSE and 43 % maximum-z improvement for learned FBL over ROM-FBL. On hardware they match the approximately 5 cm RMSE and 10 cm maximum-z error *published* by Chee et al. [3], rather than comparing the two methods on one rig; in their simulation study the flatness-based controller's maximum-z error can be worse than that of nonlinear model predictive control. The 20-fold reduction in solve time (about 1 ms against 27 ms at equal residual) is a *simulation* figure; the introduction separately quotes a sevenfold compute reduction, and the abstract an order of magnitude. Training uses 28 s of hardware data and 18 s in simulation. Two qualifications matter for our purposes: the hardware experiments use two Crazyflie 2.1 quadrotors with thrust-upgrade bundles — very nearly the platform used in this thesis — and the controller runs *off-board*, on an Intel i5 laptop streaming commands over a Crazyradio link at 400 Hz, rather than on the vehicle. Their 5 ms loop is therefore an abstract-level, off-board

figure, and the claim it supports is that the method is cheap enough to be worth porting, not that it has been shown to run onboard.

On hardware they also compensate residual *forces* only, commanding thrust and angular velocity, and delegating residual torques to the vehicle's own rate controller, the same scoping decision taken in Chapter 3. Their simulation study does still model residual torques.

Inside a model predictive control horizon. The residual can instead form part of the prediction model that an optimiser reasons over, so that the controller plans against a disturbance it expects to meet rather than cancelling the one present now. Chee et al. [3] embed a mixed-expert KNODE-DW model — first principles plus a deliberately lightweight neural ODE component — inside the prediction constraint of a nonlinear MPC, rather than adding it as an instantaneous feedforward term, and evaluate it in both simulation and hardware. The hardware experiments use two Crazyflie 2.1 vehicles in stacked formation, the lower one carrying a thrust-upgrade bundle, flown with average separations below 1.5 body lengths — one case averaging $\Delta z = 0.12\,m$, about 1.2 body lengths — with commanded separations from 0.2–0.4 m down to a very tight 0.08 m. Against a nominal MPC baseline they report average RMSE better by 40.1 % and maximum z-error smaller by 57.5 %, from 46 s of training data in total (18 s simulated and 28 s on hardware). As with the flatness-based work above, the controller runs *off-board* — an Intel i5 laptop commanding over Crazyradio at about 400 Hz with Vicon at about 120 Hz, the onboard firmware retaining PID control of thrust and body rates. Li et al. [12] take a related route, training a multilayer perceptron — spectrally normalised on its weight matrices each epoch — to predict the downwash force from the neighbour's relative position and velocity, then injecting the predicted force sequence into the nonlinear MPC's discrete prediction model as optimiser parameters, so that the lower vehicle can prepare in advance from the neighbour's planned trajectory rather than from its current state alone. Their evaluation is on hardware with two similar custom quadrotors, the optimisation running onboard a Jetson TX2 NX with pose from off-board motion capture. Hsieh et al. [7] extend the approach by inserting an $\mathcal{L}_1$ residual-acceleration estimate into the KNODE-DW prediction model, and evaluate it on *three* Crazyflie 2.1 vehicles in V-stack and I-stack formations — including a vertically aligned stack with pairwise separations of 0.2 m, about two body-lengths. Only the $\mathcal{L}_1$ estimate adapts online; the neural component reuses the training data of the work above and is not retrained in flight. The evaluation is hardware-only, again off-board, with the vehicles driven from separate machines and commanded in thrust and body rates. They report that *"the $\mathcal{L}_1$ adaptive module compensates for unmodeled disturbances most effectively when paired with an accurate dynamics model"* — in our reading, that online adaptation complements a good residual model rather than substituting for one.

Inside an optimal feedback controller for a docking task. A fourth instance is worth separating because its task differs in kind. Shankar et al. [13] embed a learned lumped downwash force online in an infinite-horizon LQR acting on a feedforward-linearised follower, beneath an inner attitude loop, and use it for vertical docking: a leader–follower manoeuvre in which the vehicles close to within one or two body-lengths, so that the disturbance is not incidental to the task but constitutive of it. The platforms are two custom quadrotors of roughly 0.27 m diagonal and 0.7 kg, not the Crazyflie class, and the free-flight dock is performed at a vertical offset of about 0.36 m — close to one body-length — with a curriculum descending to around 0.5 m. The model itself is the $SO(2)$-equivariant

one of their prior work, trained offline from about five minutes of data and queried online on an onboard Raspberry Pi, with pose from motion capture; the evaluation is hardware only. Reported accuracy depends on the manoeuvre: the abstract gives terminal error below 0.06 m, the body reports terminal vertical error typically below 0.1 m for a hovering leader, and for a moving leader below 0.05 m with the model against more than 0.2 m without it. Physical docks succeeded in eight of ten attempts against a hovering leader at 0.36 m; against a leader moving at 0.25 m/s success was limited, which the authors attribute to gripper latency. It is the closest published setting to the tightest scenarios of Chapter 5, and the regime in which — on our reading — compensation ceases to be an accuracy improvement and becomes a precondition for the manoeuvre.

2.6 Learning the compensation directly

A third stance declines to model the force at all and learns the corrective action instead. Zhang et al. [20] train a residual reinforcement-learning module on top of a cascaded controller, producing high-level commands that compensate the disturbance and thrust loss caused by downwash, and use domain randomisation to relax the requirement for accurate system identification. They train in simulation and evaluate in both simulation and hardware, on a heterogeneous pair of regular-sized quadcopters rather than the Crazyflie class. The residual and the high-level controller run *off-board* at 50 Hz, with the low-level loop onboard at 500 Hz and motion capture at 200 Hz, so this is not an end-to-end onboard residual controller.

One property distinguishes this approach sharply from everything in Section 2.4: the policy's observation comprises the tracking error, the baseline cascaded controller's command and its own previous residual action, no neighbour state, and no communication between vehicles [20]. Every relative-state method depends on knowing where the neighbours are, whether by communication or by onboard sensing. Removing that dependency removes a bandwidth requirement and a failure mode, at the cost of the ability to anticipate a neighbour the vehicle cannot perceive.

The approach also gives up the explicit force estimate that the other families produce. That estimate is what makes a predicted residual comparable against a measured one, and its absence makes failures correspondingly harder to attribute, a consideration in experimental design as much as in deployment.

2.7 Hybrid reactive and predictive compensation

Measurement and prediction are complementary in principle. Prediction is available before the disturbance acts but is valid only over the configurations its training data covered; measurement is general but delayed. Combining them is therefore natural, and has been done for a single vehicle.

Cobo-Briesewitz et al. [4] propose two methods. In the first, LINDI, a neural network replaces INDI's residual estimate entirely, trained on smoothed data obtained by spline fitting. In the second, NA-INDI, the total residual is written $f_a = f_{a,\mathrm{NN}} + f_{a,\mathrm{INDI}}$: the network predicts a component and the incremental term reasons about the remaining mismatch. Both are evaluated on a single quadrotor, with and without a slung payload, on figure-of-eight and circular trajectories, neither of which was in the training set.

The headline result concerns sensing rather than the hybrid: a network can generate smooth approximations of INDI's output without requiring rotor-speed sensors at LINDI's runtime, which removes the hardware prerequisite identified in Section 2.3 from

Table 2.1: Evaluation conditions across the interaction-force literature. Each result is reported against that work's own baseline, and no two rows share a platform, a team size, a separation and a trajectory class.

Work	Team size	Separation	Reported
Shi et al. [15]	2–4 train, 2–5 test	0.20–0.25 m vert.	max. $\|z\|$ e
Neural-Swarm2 [14]	16, heterogeneous	24 cm vert.	worst-case
Smith et al. [17]	2	0.6–0.8 m eval. (train. to ≈0.5 m)	3D and ve
Gielis et al. [5]	up to 4 total (rig sufferer + ≤ 3 flyers)	test rig, sufferer not in free flight	force-pred
Hsieh et al. [8]	2	0.3 m and 0.4 m vert.‡	RMSE an
Hsieh et al. [7]	3	V-stack and I-stack; pairwise 0.2 m	tracking v
Cobo-Briesewitz et al. [4]	1 (+ payload)	n/a	position t
Zhang et al. [20]	2, heterogeneous	0.50 m vert. hover/circle; dock to 0.10 m, release at 0.05 m	position a

† The sixteen-robot flight and the threefold reduction are separate reports; on the sixteen-robot flight the body gives a factor of about 1.5. ‡ Hardware tracking comparisons are at commanded $\Delta z = 0.3\,m$ and 0.4 m; the 0.2–0.5 m range belongs to the simulation and prediction evaluation.

that method's deployment — though rotor speeds are still needed to produce the training labels, and NA-INDI retains an incremental branch that uses them. Without a payload, residual compensation of any kind reduces tracking error by around 40 % relative to the geometric baseline; with a payload the reductions are smaller, in the range of 19–24 %. The hybrid's own advantage is real but modest: without a payload it gives the best performance, though the authors describe the margin as small, and with a payload the authors describe it as remaining *similar* to INDI alone. They also observe a slight drop in performance on the circular trajectory — for LINDI, in the no-payload case — attributing it to that trajectory being out of distribution relative to the training data, a direct instance of the limitation that defines the predictive family.

Two features of this setting bear on whether the result transfers. The residual there is self-induced, blade flapping, drag, the swing of a payload, and is predicted from the vehicle's own state. In a multi-robot team the residual is produced by other vehicles and is a function of a measurable relative state, which gives the predictive component an input the single-vehicle setting does not provide. Whether that turns a small margin into a decisive one is untested, and is one of the questions this thesis addresses.

2.8 How these methods are evaluated

The preceding sections describe a field with strong results. Reported improvements include a factor of two or better, up to four, in worst-case height error [15], up to a threefold reduction in worst-case tracking error against a zero-interaction baseline [14], 36 % and 56 % improvements in three-dimensional and vertical tracking [17], 31 % against nominal baselines as stated in the abstract [8], and roughly 40 % for residual compensation without a payload [4]. Taken individually, each is credible and internally validated.

They cannot, however, be composed. Table 2.1 lists the conditions under which they were obtained.

A 36 % improvement measured on two vehicles in close proximity and a 31 % improvement measured in a tight formation are not the same quantity: they differ in what was flown, on what airframe, at what separation, against what baseline, and under what metric. Neither can be subtracted from the other, and their ordering carries no information

about which method would prevail if both were flown under the same conditions. The difficulty compounds where the baselines themselves differ, since an improvement relative to a weaker baseline is easier to obtain.

This is not a criticism of any individual work. Each is internally valid, and there is no convention in the field that any of them violated. It is a limitation of the literature taken as a whole: the results are not designed to compose, and no amount of careful reading assembles them into a ranking. Establishing one requires an experiment built for the purpose, which is what distinguishes a comparison from a survey.

Comparisons that do exist The field does conduct systematic comparisons, and two recent ones bound what remains open.

Gu et al. [6] benchmark four online-learning uncertainty estimators — extreme learning machines, radial-basis-function networks, echo state networks and Gaussian processes — inside a single modular framework in which a nominal controller is paired with a swappable residual-learning module. They treat the uncertainty as a lumped additive wrench (f_a, τ_a) recovered by inverting the discrete Newton–Euler dynamics — analogous to the residual acceleration used here, though not identical to it a priori. They evaluate estimation accuracy, tracking performance and computational cost, and reach the kind of conclusion a comparison should: not a winner, but a set of trade-offs, with Gaussian processes most accurate yet limited by cubic training complexity, the lightweight parametric models fast but sensitive to initialisation, and echo state networks the most balanced.

Two things separate it from the present work. Its subject is a *single* quadrotor under generic uncertainty, and all four methods learn *online*; it therefore compares estimators *within* one family rather than the families of Sections 2.3 to 2.7. And its evaluation is in simulation throughout: the reported hardware component is the learning modules executing on an embedded computer in the loop with a simulator, which measures latency rather than flight, and the authors list validation on a real quadrotor as future work.

Its approach to fairness is nonetheless directly relevant, and is the opposite of the one taken here. They tune the parametric models — the extreme learning machine, the radial-basis-function network and the echo state network — to their own optima by Bayesian optimisation under otherwise consistent conditions, so that each operates near its best achievable performance; this thesis freezes a shared gain set instead. Both answer the same objection — that an observed difference might reflect tuning effort rather than method — and Section 4.10 returns to why the choice is genuinely open.

Abro et al. [1] compare fractional-order PID, fractional-order LQR and integral state feedback on close-proximity flight, explicitly motivated by downwash and by the excessive safety distances that neglecting it imposes.[2] This is a controller comparison in our setting: multiple vehicles, and downwash-aware. Of the three, only integral state feedback is classical; the fractional-order laws are not. All three are nonetheless feedback regulators: they do not present a learned or first-principles predictor of the interaction force in the sense of the works above, so the reactive, predictive and hybrid families remain uncompared against one another.

Together these delimit the gap precisely. It is not that close-proximity control is uncompared, nor that the field lacks the methodology. It is that the specific families defined by *how they obtain information about the interaction force*, measure it, predict it, or both, have not been placed under common conditions.

[2] Cited for the methods compared and for that motivation; the abstract's quantitative improvement figure and its claim about larger swarms are not used here.

Such an experiment has requirements that follow directly from the failure modes above: a single airframe; identical trajectories and formations across all methods; gains frozen before data collection and shared wherever the architecture permits, so that a difference cannot be a tuning artefact; a single definition of the residual, measured identically under every controller including those that do not use it; and a null condition in which no interaction is present, to establish the measurement floor and so distinguish a real effect from noise. Chapter 5 describes the resulting design.

2.9 Summary and gap

A controller can obtain information about the interaction force in two ways. It can measure the residual online, which requires no prior model and generalises to disturbances never seen, but is delayed and, in its established form, depends on rotor-speed sensing. Or it can predict the residual from the relative states of its neighbours using a model learned offline, which acts ahead of the disturbance but is valid only where the training data reached. A third approach learns the corrective action directly, forgoing an explicit force estimate and, in at least one instance, inter-vehicle communication as well. Representative methods exist for each, and each is individually validated.

Three gaps follow from this survey.

First, and principally, to the best of our knowledge the reactive, predictive and hybrid families have not been evaluated against one another under identical conditions. Systematic comparisons in adjacent territory exist — of online-learning estimators on a single vehicle, and of classical regulators for close-proximity swarms (Section 2.8) — but none spans the families distinguished by how they obtain the interaction force. The individual results are strong; what the field lacks is not strong results but comparable ones, and its reporting conventions — improvements against each work's own baseline, on its own platform, at its own separations — make the comparison impossible to assemble after the fact.

Second, the hybrid of measurement and prediction has been demonstrated only for a single vehicle, where the residual is self-induced and its advantage was described as small without a payload and as leaving performance similar to the reactive method with one [4]. The multi-robot case, in which the residual is larger and is a function of a measurable relative state, is where the predictive component has the most to contribute and remains untested.

Third, the summation used to aggregate per-neighbour predictions is known to be an approximation [15, 5], yet the magnitude of the penalty it carries for a deployed model, evaluated on teams larger than those it was trained on, is not reported.

Provenance, not pending work. Supported by two audits: an arXiv keyword sweep, and a cross-database sweep over OpenAlex, Crossref and DBLP including a citation-graph scan of 157 works citing the Neural-Swarm line. Both found adjacent comparisons but none spanning these families. Neither is exhaustive; keep the "to the best of our knowledge" phrasing.

Chapter 3 formalises the residual wrench these methods estimate or predict, and Chapter 4 describes the strategies compared in this thesis and the point at which each injects its knowledge of the interaction into the control law.

Chapter 3

System Modelling and Residual-Force Formulation

Chapter 2 surveyed families that measure a residual, predict one, plan over one, or learn a corrective action with no force estimate at all. Those families are comparable as *controllers*, on the same vehicles, trajectories and formations. They are not required to compute the same internal quantity.

This chapter defines the residual acceleration a_{res} that *this* stack measures and logs. Two things follow. Strategies 2, 3 and 5, and any other method that consumes a force residual, then refer to one number rather than to several differently-defined ones. And a predicted residual can be checked against a measured one on the same flight, which is how a learned model is evaluated here. Residual reinforcement learning (Strategy 7) never forms that number. That is a property of the method, not a reason to exclude it from a comparison of performance.

The chapter also states the assumptions the formulation rests on, and where each one is expected to fail. Several are known to be imperfect, the summation in Section 3.4 most conspicuously, and stating them here is what allows the discussion chapter to interpret a negative result as evidence about an assumption rather than as an unexplained failure.

3.1 Notation and conventions

Vectors are expressed in a world frame that is *east-north-up*: the third basis vector e_3 points away from the ground and gravity acts along $-e_3$. This matches the convention used by the flight software and the motion-capture system, and is stated explicitly because the opposite convention is common in the quadrotor literature and the sign of the gravity term differs between them.

For vehicle i: $p_i \in \mathbb{R}^3$ is position, $v_i \in \mathbb{R}^3$ velocity, $R_i \in SO(3)$ the rotation from body to world, and $\omega_i \in \mathbb{R}^3$ the body angular velocity. The hat map $\hat{\cdot} : \mathbb{R}^3 \to \mathfrak{so}(3)$ is defined by $\hat{a}b = a \times b$. The vehicles are homogeneous, of mass m and diagonal inertia J. Rotor k spins at ϖ_k and produces thrust $k_{t,k}\varpi_k^2$ along the body z-axis.

3.2 Rigid-body model with a residual wrench

Each vehicle is modelled as a rigid body driven by a collective thrust f_i along its body z-axis and a body torque τ_i:

$$\dot{p}_i = v_i, \qquad m\,\dot{v}_i = -mge_3 + f_i R_i e_3 + f_{\mathrm{res},i}, \tag{3.1}$$

$$\dot{R}_i = R_i \hat{\omega}_i, \qquad J\dot{\omega}_i = -\omega_i \times J\omega_i + \tau_i + \tau_{\mathrm{res},i}. \tag{3.2}$$

The terms $f_{\mathrm{res},i}$ and $\tau_{\mathrm{res},i}$ form the *residual wrench*: by construction they collect every force and moment that (3.1)–(3.2) otherwise omits. This includes blade flapping, rotor drag, ground effect and errors in m, J and k_t, as well as the inter-vehicle aerodynamic interaction that is the subject of this thesis.

That the residual is a catch-all rather than a physical quantity has two consequences that matter for the experiments. First, a residual measured in isolated free flight is not zero; it is whatever the nominal model gets wrong about a single vehicle. Second, the interaction can only be isolated by *differencing*, comparing the residual measured in formation against the residual measured alone under the same trajectory. The coplanar control scenarios in Chapter 5 exist for exactly this purpose.

Scope: force, not moment This thesis compensates the *force* residual f_{res}. The moment residual τ_{res} is measured and logged, but no strategy compensates it separately: the attitude loop rejects it as a disturbance in the usual way. This is a scoping decision, not a claim that τ_{res} is negligible.

The decision has company. Neural-Swarm compensates the vertical interaction force [15], and the hardware controller of Hsieh et al. [8] likewise commands thrust and angular rates, leaving residual torque to the rate loop. Kiran et al. [9] and Gielis et al. [5] do measure both force and moment, but on a rig rather than as a flight residual.

We therefore log τ_{res} and do not cancel it as a separate term. The cost is that interaction torque is left to the attitude loop. The benefit is a single compensation channel that fits the existing allocator and the onboard budget. The discussion chapter returns to what leaving it uncompensated may cost.

3.3 Measuring the residual

The residual can be recovered without any model of the interaction, by comparing what the vehicle actually accelerated with what the nominal model says it should have accelerated given the actuation actually applied. Rearranging (3.1),

$$\frac{f_{\mathrm{res},i}}{m} = \underbrace{\dot{v}_i}_{a_{\mathrm{meas}}} - \underbrace{\left(\frac{f_i}{m} R_i e_3 - g e_3\right)}_{a_{\mathrm{model}}}. \tag{3.3}$$

Both terms are available onboard. The measured acceleration comes from the accelerometer, which senses specific force in the body frame and therefore excludes gravity by construction; rotating it into the world frame and subtracting ge_3 gives a_{meas}. The model acceleration requires the applied thrust, which is reconstructed from measured rotor speeds as $f_i = \sum_{k=1}^{4} k_{t,k}\varpi_k^2$.

The implementation evaluates (3.3) at every control step and logs it as a_{res}, in every controller mode, including modes that make no use of it. This is deliberate. The training data for every predictive strategy is collected under the *uncompensated* geometric controller, so the measurement must exist when no INDI term is active; and the comparison

of a predicted residual against a measured one, which is how a learned model is evaluated, requires both quantities on the same flight.

3.3.1 What this measurement requires, and how it fails

Three properties of (3.3) shape the experimental design.

It requires rotor-speed sensing. Without ϖ_k the applied thrust is unknown and a_{model} cannot be formed. This is the hardware prerequisite identified in Section 2.3, and the same one that Cobo-Briesewitz et al. [4] remove by learning the residual instead of computing it. In the implementation, the absence of a rotor-speed source causes a_{res} to read *identically zero*. That failure is silent and is indistinguishable, in a log file, from a flight in which no interaction occurred. Verifying that $a_{\text{res}} \neq 0$ is therefore the first check of every experimental session (Chapter 5).

It is delayed. a_{meas} is differentiated from noisy inertial measurement and must be filtered; the reconstructed thrust inherits the rotor-speed sensing delay. The estimate therefore exists only after the disturbance has begun to act. No amount of tuning removes this: it is the structural property that motivates prediction.

It absorbs model error. Any error in m or k_t appears in a_{model} and is attributed to f_{res}. A miscalibrated thrust constant produces a residual that looks like a constant interaction force. This is why the airframe constants are identified once and frozen, and why the null scenarios matter: a residual that persists when no neighbour is present is model error, not interaction.

3.3.2 Sign convention in the control law

The compensation enters with a sign that is easy to get wrong and whose error is not obvious in flight. From (3.1), holding a desired acceleration a_{des} requires

$$f_i R_i e_3 \;=\; m\, a_{\text{des}} + mge_3 - f_{\text{res},i}, \tag{3.4}$$

so the desired-acceleration vector passed to the thrust-and-attitude allocation carries $-f_{\text{res},i}/m$. The residual is *subtracted*, whether it was measured (Section 3.3) or predicted (Section 3.4).

Adding it instead does not merely fail to compensate: it doubles the disturbance, because the controller then commands a thrust error equal and opposite to the correction it should have made. The resulting behaviour is a stable, well-controlled vehicle that tracks slightly worse in formation, which is what an uncompensated vehicle also looks like. In single-vehicle flight the error is invisible altogether, since $f_{\text{res}} \approx 0$ with no neighbour present. This is recorded here because it is a property of the formulation rather than a coding detail, and because it establishes a specific prediction that the two-robot results chapter can check: with the sign correct, position-INDI compensation must *reduce* the formation-induced displacement, and with it inverted the displacement must be approximately twice the uncompensated value.

3.4 Predicting the residual from relative state

The interaction force on vehicle i is a function of the configuration of the team relative to it,

$$f_{\mathrm{res},i} \;\approx\; \mathcal{F}(\{\, p_j - p_i,\; v_j - v_i \,\}_{j\neq i}) \,, \tag{3.5}$$

and (3.5) is what a predictive strategy learns. Two structural properties are required of any parameterisation of $\mathcal{F}$, and both follow from the physics rather than from convenience. The value must not depend on the order in which neighbours are enumerated, and it must be defined for a varying number of them, since a vehicle in a three-robot team has two neighbours and one in a pair has one.

3.4.1 Deep-sets parameterisation

The parameterisation used here satisfies both by construction:

$$\hat{a}_{\mathrm{res},i} \;=\; \rho\left(\sum_{j\neq i} \phi(p_j - p_i,\; v_j - v_i)\right) , \tag{3.6}$$

where ϕ is applied to each neighbour independently and ρ maps the summed embedding to a residual acceleration in the world frame. Summation is permutation-invariant and accepts any number of terms, so one parameter set serves teams of different sizes. The output is an acceleration rather than a force so that it is directly comparable with the measured a_{res} of (3.3); the two quantities are in the same units, in the same frame, at the same point in the control loop.

What is borrowed and what is ours. The *family* is the permutation-invariant deep set used by Neural-Swarm and Neural-Swarm2 [15, 14]. The layer widths and the resulting 987 parameters are ours, sized for the STM32 control period. This is not the network of Cobo-Briesewitz et al. [4], and it is not a copy of Neural-Swarm2's layers. Spectral normalisation is not used here. An architecture ablation would be a separate study and is out of scope unless time remains after the comparison.

The concrete network is small: ϕ maps $6 \to 16 \to 16 \to 8$ and ρ maps $8 \to 16 \to 16 \to 3$, with rectified-linear activations throughout except at the output, which is linear because an interaction force may point either way. This is 987 parameters, evaluated onboard within the control period. The sizing is discussed in Chapter 4; what matters here is that (3.6) is the model, and the layer widths are an implementation choice within it.

These equations are the code. Equation (3.6), the layer widths above, the distance guards of Section 3.4.3 and the output bound are the values in the residual-network source file,[1] not an idealisation of them, and were read from it rather than reconstructed. The same applies to (3.3) and the sign convention of Section 3.3.2. If the implementation changes, this chapter changes with it; a modelling chapter that has drifted from the flight code describes a system that was never flown.

[1] `firmware_app/src/residual_nn.rs` for the network; `firmware_app/src/lib.rs` for the residual computation and the desired-acceleration assembly.

3.4.2 The superposition assumption

Equation (3.6) assumes the aggregate interaction is a sum of per-neighbour contributions in the latent space of ϕ. This is more general than summing forces directly, ρ is free to be nonlinear in the sum, but it is still an assumption, and the literature states plainly that it does not hold exactly. Shi et al. [15] report that with more than two vehicles the aerodynamic effect is not a simple superposition of each pair, and Gielis et al. [5] motivate a learned aggregation model, measured on a load stand, precisely on the grounds that a single-vehicle downwash model is unlikely to suffice for predicting aggregate forces.

The assumption is retained here for two reasons. It is what makes one trained model applicable to teams of two and three without retraining, which the comparison requires. And a controller does not need an exact model: it needs one whose residual after compensation is smaller than the residual before it. The relevant question is therefore not whether (3.6) is exact, which it is not, but how large the resulting error is on a deployed model, which is what the multi-robot results chapter measures.

3.4.3 Input conditioning

Two guards are applied to the inputs of ϕ, and both are part of the model rather than defensive programming, because they change the function being learned.

A neighbour farther than 2 m is excluded from the sum. Beyond that distance the interaction is below the noise floor of the measurement, and including such neighbours would ask the network to learn that distant vehicles do nothing, spending capacity on a region where the answer is known.

A separation closer than 4 cm is clamped to that value, preserving direction. Below it the model would extrapolate beyond anything the training data can contain, since the vehicles are 9 cm across and such a configuration is a collision. A learned function is least trustworthy exactly where the physics is strongest, and the clamp bounds the extrapolation rather than trusting it.

The output is bounded as well, at $8\,\mathrm{m/s^2}$, comfortably above the interaction accelerations expected between vehicles of this size, and far below what an ill-conditioned network can emit. This is not a hypothetical failure mode. In simulation, Gu et al. [6] report a radial-basis-function estimator crashing the vehicle under periodic wind, and an extreme learning machine becoming unstable on landing under ground effect; their echo state network and Gaussian process did not. The authors say the failure *may be attributed* to the random initialisation of the input-to-hidden weights, which *"can produce excessively large prediction values"*. Their vehicle is a single quadrotor and their disturbances are wind and ground effect, so this is not evidence about inter-vehicle interaction, but it is evidence that an unbounded learned term can destabilise a controller. A saturating output converts that failure into a bounded and, because saturation is logged, a visible one.

The training data must pass through the same guards as the deployed model. A model fitted to unguarded inputs is fitted to a different distribution than the one it will meet, and the discrepancy is invisible until predicted and measured residuals are compared in flight.

3.4.4 Normalisation

Input scaling is folded into the first layer of ϕ at export rather than applied onboard. For weights W_1 and bias b_1 trained on inputs normalised by mean μ and standard deviation

Table 3.1: Assumptions of the residual formulation and their expected failure modes.

Assumption	Where it fails	Consequence
Rigid body; no aeroelastic deformation	Not expected to fail at this scale	—
m, J, k_t known and constant	Battery discharge; rotor wear; miscalibration	Model error appears as a constant residual; bounded by the null scenarios
Residual is dominated by inter-vehicle interaction	Near the ground; near walls	Flight volume is bounded away from both (Chapter 5)
Neighbour relative states are known accurately	Motion-capture dropout; latency	Prediction degrades; measurement is unaffected, which is itself a comparison result
Aggregate interaction superposes in latent space	Three or more vehicles, overlapping wakes	**Known to be imperfect** [15, 5]; magnitude measured in the multi-robot results chapter
Interaction lies within the training distribution	Unseen separations, faster relative motion	Prediction degrades while measurement does not; this is the crossover of **RQ2**
Rotor-speed measurement available	No RPM source fitted	a_{res} reads identically zero — silent, and checked every session

σ,

$$W_1 \frac{x-\mu}{\sigma} + b_1 \;=\; \underbrace{(W_1 \oslash \sigma)}_{W_1'}\, x + \underbrace{(b_1 - (W_1 \oslash \sigma)\,\mu)}_{b_1'}, \tag{3.7}$$

with $\oslash$ denoting column-wise division. The two forms are arithmetically identical, so the deployed network computes the same function as the trained one while the flight software carries no normalisation constants that could fall out of step with the model.

3.5 Assumptions and where they fail

Table 3.1 collects the assumptions this formulation makes. Each is stated with the condition under which it is expected to break, so that an experimental result can be attributed to a specific assumption rather than to the model as a whole.

Two entries deserve emphasis because they are not merely caveats but hypotheses the experiments test. The superposition assumption is known to be imperfect, and the experimental question is the size of the penalty rather than its existence. And the training-distribution assumption is exactly what separates the predictive family from the reactive one: a measurement has no training distribution to leave. Cobo-Briesewitz et al. [4] observe precisely this effect for their purely learned method in the no-payload case, attributing a drop in performance on the circular trajectory to its being out of distribution relative to the training data.

3.6 Summary

The residual of (3.1)–(3.2) is what this stack measures and logs. Strategies 2, 3 and 5 consume it. Strategy 7 never forms it, which is a property of residual reinforcement learning rather than a reason to exclude it from a comparison of performance. The residual

can be recovered online from (3.3), which needs no model of the interaction but requires rotor-speed sensing and is available only after the disturbance acts; or predicted from (3.6), which acts ahead of the disturbance but is valid only within its training distribution and assumes an aggregation that is known to be approximate. Both are expressed as accelerations in the world frame and enter the control law at the same point with the same sign, (3.4).

That common definition is what makes the comparison in the two-robot results chapter a comparison rather than a collection of separately-tuned controllers. Chapter 4 describes the strategies built on it, and the point at which each injects its knowledge of the interaction.

Chapter 4

Control Architectures

Chapter 3 defined the residual wrench and the two ways of obtaining it. This chapter describes the control strategies compared in this thesis, and in each case the point at which knowledge of the interaction enters the control law.

Strategies 1–6 plus 7a and 7b (eight controllers; seven families, with residual reinforcement learning split by inner loop). Table 4.1 lists them, and the rest of the chapter takes them in order.

The chapter is organised around a single observation. The strategies share far more than they differ by. The fair set, Strategies 1, 2, 3 and 5, tracks the same trajectories on the same airframe through the same attitude allocation; Strategies 4, 6 and 7 do not share that allocator. What separates them is which terms are present in one vector, the desired acceleration of the position loop, and where that vector's information comes from. Presenting them this way is not a simplification for exposition. It is the structural property that makes the comparison fair, and Section 4.10 states precisely what it does and does not guarantee.

Each strategy is described in two parts, kept deliberately separate: what is implemented here, and the published method it follows.

Core Strategies 1, 2, 3 and 5 run on one geometric-plus-optional-INDI substrate, so that the comparison isolates how interaction information enters a_{des}. They follow published *families*, and the deviations from each source paper are listed in that strategy's section. They are not bit-identical replicas. Optional Strategies 4, 6 and 7 aim for closer fidelity to their sources if author code arrives; until then they stay unimplemented rather than guessed.

4.1 The common substrate

On *this* stack, Strategies 1, 2, 3 and 5 share a geometric position and attitude substrate. The position loop forms a desired acceleration and the attitude loop realises the corresponding thrust direction. Strategy 2 replaces the attitude law with an incremental one. Strategies 4, 6 and 7 use different published structures, namely feedback linearisation, receding-horizon optimisation and residual reinforcement learning, and therefore do not share that substrate.

Table 4.1: The strategies compared, and the implementation status of each at the time of writing. "Knows" refers to the residual: whether the controller has a measurement of it, a prediction of it, both, or neither.

#	Strategy	Knows	Enters at	Rotor speed	Status
1	Geometric baseline	nothing	—	no	flying
2	Pure INDI	measured	a_{des}, torque	yes	flying
3	Geometric + NN	predicted	a_{des}	no	implemented, unflown
4	FBL + NN	predicted	linearising law	no	not implemented
5	Hybrid NA-INDI	both	a_{des}, torque	yes	form fixed, not wired
6	Learning-based MPC	predicted	MPC prediction model	no	not implemented
7a	Residual RL, cascaded	neither	actuation	no	deferred
7b	Residual RL, geometric	neither	actuation	no	deferred

4.1.1 Position loop

The desired acceleration is

$$a_{\text{des}} = \underbrace{a_{\text{ff}}}_{\text{trajectory}} + K_p e_p + K_v e_v + K_i \int e_p \, \mathrm{d}t + g e_3 \underbrace{- a_{\text{res}}}_{\text{measured}} \underbrace{- \hat{a}_{\text{res}}}_{\text{predicted}}, \tag{4.1}$$

with $e_p = p_{\text{des}} - p$ and $e_v = v_{\text{des}} - v$, and gains diagonal but distinct in the horizontal and vertical axes. The commanded thrust vector is $m\, a_{\text{des}}$; its magnitude projected onto the body z-axis gives the collective thrust, and its direction, together with the desired yaw, gives the desired attitude R_{des} through the usual flatness construction.

The last two terms of (4.1) are the whole subject of this thesis, and both are subtracted, for the reason derived in Section 3.3.2. *Which residual information is present is what distinguishes Strategies 1, 2, 3 and 5 from one another*, and nothing else about the position loop changes between them: Strategy 1 has neither term, Strategy 2 the measured one, and Strategy 3 the predicted one.

Strategy 5, which has both, does not simply enable the two terms together. It uses the predicted residual and then corrects with the *difference* between measurement and prediction, for the reason given in Section 4.6; (4.6) states that form. Written as (4.1) with both terms active, the two would double-count whatever they capture in common.

4.1.2 Attitude loop

The attitude error is the standard geometric one, $e_R = \frac{1}{2}\left(R_{\text{des}}^\top R - R^\top R_{\text{des}}\right)^\vee$, with $e_\omega = \omega - \omega_{\text{des}}$. In the geometric configuration the torque is

$$\tau = -K_R e_R - K_\omega e_\omega + \omega \times J\omega - k_{i,\text{att}} \int e_R \, \mathrm{d}t, \tag{4.2}$$

the third term compensating the gyroscopic coupling. This is a reduced form of the law in [11]: it drops that work's angular-feedforward bracket and adds an integral term the source does not have — see (4.3) and the discussion in Section 4.2. The INDI configurations replace (4.2) with an incremental law, described in Section 4.3.

4.1.3 Runtime selection

The control law is selected by a parameter at runtime rather than by recompiling: one bit enables the measured residual in the position loop, another selects the incremental

attitude law, and a third parameter enables the learned prediction. All are pushed to the vehicle at connection time from a single configuration file.

This matters more than it may appear. Comparing strategies that require different firmware introduces a confound that is impossible to rule out afterwards, a flash cycle changes more than the control law, and strategies flown on different days meet different battery states, different temperatures and different motion-capture calibrations. Runtime selection allows the strategies to be interleaved within a single session, which is what makes the comparison a paired one.

4.2 Strategy 1: uncompensated geometric baseline

Implemented. Equations (4.1) and (4.2) with both residual terms zero. No knowledge of the interaction whatsoever.

Reference. Geometric tracking on $SE(3)$ [11], which tracks a position trajectory together with a single heading direction, four outputs, not a full six-degree-of-freedom pose. The attitude error of (4.2) is that work's $e_R = \frac{1}{2}(R_d^\top R - R^\top R_d)^\vee$, verified against the source rather than reproduced from memory.

The implemented torque law is a reduced form, and is labelled as such. The law in the source is

$$M = -k_R e_R - k_\Omega e_\Omega + \Omega \times J\Omega - J\Big(\hat{\Omega}\, R^\top R_d \Omega_d - R^\top R_d \dot{\Omega}_d\Big)\,, \tag{4.3}$$

with $e_\Omega = \Omega - R^\top R_d \Omega_d$. Equation (4.2) retains the proportional, damping and gyroscopic terms but omits the final feedforward bracket, and adds an integral term on attitude error that the source does not contain. It is therefore a *reduced* law in the same family, not the torque law of [11], and no stability result proved there transfers to it unexamined. Restoring the feedforward bracket would be a firmware change rather than a wording change, and is outside the scope of this draft.

What the source proves. Not global asymptotic tracking. With the attitude error function Ψ, exponential stability is established on $\Psi(0) < 1$, initial attitude errors below $90\,^\circ$, and almost-global exponential attractiveness on $1 \le \Psi < 2$, below $180\,^\circ$. The distinction matters here only in that this thesis claims neither: the baseline is flown as a reference condition, not as a certified controller.

Role. Strategy 1 is the uncompensated baseline of Table 4.1. It is flown in every scenario so that "how much did compensation help" has a denominator, and so that the interaction can be separated from the model error discussed in Section 3.3.1. It is also the controller under which all training data is collected, precisely because it does not compensate: a residual observed under a compensating controller is the residual that survived compensation, which is not what a predictive model should be fitted to.

4.3 Strategy 2: pure INDI

Implemented. The measured residual of (3.3) enters the position loop as $-a_{\text{res}}$ in (4.1). In the attitude loop the geometric torque law is replaced by an incremental one. A reference

angular acceleration is formed from the geometric errors,

$$\alpha_{\text{ref}} = \alpha_{\text{des}} - K_R e_R - K_\omega e_\omega, \tag{4.4}$$

where α_{des} is the feedforward angular acceleration obtained from trajectory snap by differential flatness. The achieved angular acceleration α_{meas} is obtained by differentiating the gyro signal and low-pass filtering it, and the torque currently being applied, τ_{cur}, is reconstructed from measured rotor speeds. The command is then the increment

$$\tau = \tau_{\text{cur}} + J\left(\alpha_{\text{ref}} - \alpha_{\text{meas}}\right). \tag{4.5}$$

Because (4.5) corrects relative to what is actually being achieved, it rejects any disturbance appearing in α_{meas} without needing a model of it. The differentiation and filtering are the source of the delay identified in Section 3.3.1; the filter cutoff sets the trade-off between that delay and the noise admitted from the gyro.

The position and attitude increments are independently selectable, giving four configurations: geometric, position-INDI only, attitude-INDI only, and full INDI. Strategy 2 refers to the full configuration, and that is the controller the comparison reports. The intermediate ones are diagnostic bits used in the two-robot results chapter to attribute an effect to one loop or the other. They are not additional strategies and will not be reported as Strategy 2.

Reference. The incremental formulation with flatness-derived feedforward follows Tal and Karaman [18]; the cascaded arrangement of position and attitude INDI follows Smeur et al. [16].

Deviations. The airframe, gains and filter cutoffs are ours and were identified for this platform. Tal and Karaman obtain rotor speed from optical encoders [18]; this implementation uses the platform's own rotor-speed measurement. No claim is made that this implementation reproduces the tracking performance reported in either work, that would require a like-for-like replication which is not part of this thesis.

Requirements. Rotor-speed sensing, without which τ_{cur} and a_{res} are both unavailable and the method degrades silently (Section 3.3.1). The method needs no training data; the residual is formed online from inertial and rotor-speed measurements.

4.4 Strategy 3: geometric with a learned residual

Implemented. The geometric controller of Section 4.2 with the deep-sets prediction $\hat{a}_{\text{res}}$ of (3.6) subtracted in (4.1). The network is evaluated onboard at the control rate from the relative states of the neighbours, which the flight stack already broadcasts, so the input requires no additional communication. Weights are trained offline on data collected under Strategy 1 and uploaded before flight.

The prediction is computed and logged *whether it is enabled or not*. This is what makes the model's own accuracy measurable independently of its effect on the vehicle: on a flight where the prediction is not driving the controller, predicted and measured residuals are two independent quantities and their comparison is meaningful. Once the prediction is driving the controller, the measured residual is the one that survived compensation and the comparison no longer isolates model quality. Both flights are therefore needed, and Chapter 5 schedules them.

Reference. The approach of predicting the interaction from relative states with a permutation-invariant network and applying it as feedforward follows the Neural-Swarm line [15, 14].

Deviations. These are substantial and are stated plainly. The architecture here is a deep-sets model of our own sizing, 987 parameters, rather than a reproduction of any published network. Neural-Swarm2 applies spectral normalisation to obtain stability and generalisation guarantees [14]; this implementation does not, and the guarantees that follow from it are correspondingly not claimed. Neural-Swarm2 also uses the learned residual in motion planning as well as control [14], whereas here the trajectories are planned in advance and the residual is used for control only. This strategy is a representative of the predictive family, and should be read as such rather than as an evaluation of Neural-Swarm2.

Requirements. Neighbour relative states; training data. No rotor-speed sensing.

4.5 Strategy 4: feedback linearisation with a flatness-preserving residual

Implemented. Not implemented. The controller code was requested from the authors and had not been received at the time of writing. The strategy does not block the others, and its absence is a scope limitation rather than a failure of the comparison (the discussion chapter).

Reference. Hsieh et al. [8], building on the theory of Yang et al. [19]. The distinguishing property is where the prediction enters: not as a feedforward term added to a desired acceleration, but inside a feedback-linearising law whose validity depends on the augmented system remaining differentially flat. A generic learned residual can destroy that flatness; residuals with a lower-triangular structure, a structured correction inside the state derivative, not a free wrench, preserve it, and with it the original flat output [19]. That theory is established in simulation on a planar quadrotor; the hardware demonstration is Hsieh et al.'s.

Its relevance to this comparison is that it is the strongest published counter-argument to treating "predictive compensation" as one method. Hsieh et al.'s abstract reports a 31 % reduction in average tracking error; on hardware the body reports 24 % RMSE for ROM-FBL over nominal FBL and a further 29 % for learned FBL over ROM-FBL, matching the tracking error published by Chee et al. for nonlinear model predictive control, at an order of magnitude less computation in simulation, from under 30 s of training data, and at a 5 ms loop rate obtained *off-board* [8]. Those are the figures a predictive strategy in this class should be measured against, with the caveat that none of them was produced by a controller running on the vehicle.

4.6 Strategy 5: hybrid neural-augmented INDI

Form adopted. The hybrid follows the *reference* arrangement. The network predicts the bulk of the residual and the incremental term corrects only what the network leaves behind, so that the desired acceleration carries

$$a_{\mathrm{des}} = a_{\mathrm{ff}} + K_p e_p + K_v e_v + K_i \int e_p \,\mathrm{d}t + g e_3 \underbrace{-\,\hat{a}_{\mathrm{res}}}_{\text{prediction}} \underbrace{-\,(a_{\mathrm{res}} - \hat{a}_{\mathrm{res}})}_{\text{what the prediction missed}} , \tag{4.6}$$

rather than $-a_{\text{res}} - \hat{a}_{\text{res}}$.

Subtracting both terms independently was considered and rejected. It double-counts whatever portion of the disturbance the two capture in common, so a network that predicted *well* would produce over-compensation rather than better compensation, and the strategy would then be measuring an artefact of the arrangement instead of the value of combining the two information sources. Equation (4.6) degrades gracefully at both limits: with a perfect prediction the incremental term vanishes and the strategy reduces to Strategy 3, and with no prediction it reduces exactly to Strategy 2.

The cost of the choice is stated rather than hidden. Because the incremental term absorbs whatever the network misses, a poor prediction is partly masked, and the hybrid's tracking performance alone cannot reveal how good the model is. This is why the predicted and measured residuals are logged separately on every flight (Section 4.4): model quality is assessed from $\hat{a}_{\text{res}}$ against a_{res} on a flight where the prediction is not driving the vehicle, never inferred from how well the hybrid tracks.

Implemented. Not yet wired. Both components exist and are individually selectable, the measured residual through the position-INDI bit, the prediction through `rnn.en`, but the differencing in (4.6) is not implemented. It is a change to the control law and belongs after the hardware gate of Chapter 5.

Reference. Cobo-Briesewitz et al. [4], who propose both a purely learned replacement for the incremental estimate and a hybrid, NA-INDI, in which the residual is written $f_a = f_{a,\text{NN}} + f_{a,\text{INDI}}$ and the incremental term reasons about the remaining mismatch left by the network.

What their result does and does not establish. On a single quadrotor *without* a slung payload, residual compensation of any kind reduced tracking error by around 40 % relative to the geometric baseline; with a payload the reductions are smaller, in the range of 19–24 %. The hybrid gave the best performance without a payload, though the authors describe the margin as small, and with a payload the authors describe it as remaining *similar* to the incremental method alone. Their headline result is a different one: the learned model generates smooth approximations of the incremental estimate *without rotor-speed sensing* at runtime, rotor speeds are still required to produce the training labels, and the hybrid's incremental branch still uses them [4].

The setting differs from ours in a way that bears directly on whether the hybrid's small margin transfers. There the residual is self-induced, their introduction names blade flapping, drag and ground forces, and is predicted from the vehicle's own state; in the payload case the network takes a 28-dimensional input comprising vehicle and payload velocity and acceleration and the cable direction, with the cable tension itself belonging to the nominal payload model rather than to the residual. Here it is produced by other vehicles and predicted from a measurable relative state: a larger disturbance, and a more informative input for the predictive half. Whether that widens the margin is the question, and a negative answer is equally informative, since it would indicate that the reactive term alone suffices and the training campaign is not worth its cost.

Requirements. Rotor-speed sensing *and* neighbour relative states *and* training data, the union of Strategies 2 and 3, which is itself a result for the cost accounting of the discussion chapter.

4.7 Strategy 6: learning-based model predictive control

Implemented. Not implemented. Whether it can run within the control period on this platform is an open engineering question, and one that is itself relevant. This is the only strategy in the comparison whose computational cost may exclude it from the target hardware. Nothing in the literature settles it either way for our vehicle: Chee et al. and Hsieh et al. both run their optimisation off-board, and Li et al. run theirs on a Jetson companion computer, not on an STM32.

Reference. Chee et al. [3] embed a mixed-expert KNODE-DW model — first principles plus a lightweight neural ODE component — inside the prediction constraint of a nonlinear MPC, and evaluate it in simulation and on hardware with two Crazyflie 2.1 vehicles in stacked formation at average separations below 1.5 body lengths, with the controller running off-board. Hsieh et al. [7] extend this by inserting an $\mathcal{L}_1$ residual-acceleration estimate into the same prediction model, evaluated on hardware with *three* Crazyflie 2.1 vehicles in V-stack and I-stack formations at pairwise separations down to 0.2 m, and report that the adaptive module is most effective when paired with an accurate dynamics model. Only the $\mathcal{L}_1$ estimate adapts online.

Where the prediction enters. Not at an instant, as in Strategies 3 and 5, but inside the prediction model over which the optimiser reasons. The controller therefore plans against a disturbance it expects to meet rather than cancelling the one present now, the most distinct injection point in the comparison, and the reason the strategy is retained in the set even though it may prove infeasible onboard.

4.8 Strategy 7a: residual reinforcement learning on a cascaded baseline

Implemented. Not implemented, and deliberately scheduled last.

Reference. Zhang et al. [20] train a residual reinforcement-learning module on top of a *cascaded* controller, not a geometric one. It produces corrections that compensate the disturbance and thrust loss caused by downwash, and uses domain randomisation to reduce the dependence on accurate system identification. Their residual and high-level controller run off-board at 50 Hz above an onboard low-level loop at 500 Hz, on regular-sized quadcopters. Strategy 7a follows that arrangement.

4.9 Strategy 7b: residual reinforcement learning on the geometric baseline

Implemented. Not implemented. This pairing is ours, not any paper's.

What it shares with Strategy 7a. The stance is identical. A policy outputs a corrective action, and no explicit force estimate is formed at any point. Both are therefore evaluated on tracking performance alone, and neither can be checked by comparing a predicted residual against a measured one.

What differs. The inner loop. Strategy 7a sits on the cascaded controller of [20]; Strategy 7b sits on the geometric position and attitude substrate used by Strategies 1, 2, 3 and 5 in this thesis. Separating them matters because a residual policy is trained against whatever the baseline leaves uncorrected, so the same learning stance on a different inner loop is a different experiment, not a re-run.

Why they are scheduled last, stated plainly. They answer a different question. Strategies 2–6 all reason about an explicit force: each produces, or consumes, an estimate of f_{res} that can be compared against the measurement of (3.3). A residual policy produces a corrective action and never forms such an estimate, so the predicted-versus-measured comparison that underpins the evaluation of every other learned strategy is unavailable, and failures are correspondingly harder to attribute.

One property nonetheless makes them worth retaining. The policy of Zhang et al. observes only the tracking error, the baseline controller's command and its own previous residual action, no neighbour state, and no communication between vehicles [20]. Every other predictive strategy here depends on knowing where the neighbours are. Removing that dependency removes a bandwidth requirement and a failure mode, at the cost of being unable to anticipate a neighbour the vehicle cannot perceive, a trade-off that belongs in the cost accounting even if the strategies are not flown.

4.10 What makes the comparison fair, and what it does not guarantee

What is held constant. The airframe, the trajectories, the formations, the residual definition, the estimator, and the gains, which are frozen before any data collection and shared across strategies wherever the architecture permits. Strategies 2, 3 and 5 additionally share the entire position and attitude loop, differing only in which of the two residual terms in (4.1) is active. For those three, a difference in outcome cannot be attributed to anything but the residual term, which is the strongest form of the fairness argument available here.

What this does not guarantee. Three limitations are worth stating before results are presented rather than after.

Core Strategies 1, 2, 3 and 5 follow published *families* on one substrate, with the deviations listed per strategy; they are not bit-identical replicas. Optional Strategies 4, 6 and 7 aim for closer fidelity if author code arrives, and stay unimplemented rather than guessed until then. Strategies 4 and 6 do not share the substrate: a feedback-linearising law and a receding-horizon optimiser are different controllers, not different terms in one controller. For those, "identical conditions" means identical trajectories, formations and hardware, not an identical control structure. The comparison is correspondingly weaker, and any difference could in principle be attributed to the controller rather than to the compensation.

Freezing shared gains is fair in the sense that no strategy was tuned more than another, but it is not necessarily favourable to all of them equally: gains chosen to suit a geometric loop may disadvantage a strategy that could tolerate more aggressive ones.

The alternative is to tune each strategy to its own optimum, and it is a real position rather than a straw man: Gu et al. [6] take exactly that route in their comparison of online-learning estimators, using Bayesian optimisation on the parametric models so that

each operates near its best achievable performance under otherwise consistent conditions. The two choices trade different risks. Per-method tuning risks measuring tuning effort, and makes the result contingent on the optimiser and its budget; shared gains risk penalising a strategy the operating point does not suit. This thesis freezes shared gains because Strategies 2, 3 and 5 share the entire control loop, which makes a shared operating point meaningful in a way it would not be across structurally different controllers, and because the tuning budget available here is one lab session, not an automated search. the discussion chapter reports where a frozen gain appears to have disadvantaged a strategy.

4.11 Summary

Seven strategies, distinguished by what they know about the interaction force, when they know it, and where that knowledge enters the control law. Four of them, the baseline, pure INDI, the learned feedforward and their hybrid, are terms in a single shared control law, and their comparison is correspondingly tight. Three others inject the same information at structurally different points, and are compared under identical conditions but not identical structure.

At the time of writing, two strategies fly, one is implemented and unflown, one exists as separable components, and three are not implemented. Chapter 5 describes the experimental design; the two- and multi-robot results chapters report what was flown.

Chapter 5

Experimental Setup and Protocol

Chapter 4 described Strategies 1–6 plus 7a and 7b (eight controllers; seven families, with residual reinforcement learning split by inner loop), and argued that four of them differ only in which residual term is active. That argument is only worth as much as the experiment that exercises it. This chapter describes the platform, the scenarios, the measurement chain and the protocol, and in each case the reasoning is the same: the design exists to remove alternative explanations for an observed difference.

Section 5.7 states the threats this design is built against, and which of them it does not eliminate.

5.1 Platform

The vehicles are Crazyflie 2.1 brushless quadrotors, nominally 41 g and 9 cm rotor-to-rotor. All vehicles in a team are the same platform: the gain block is shared across robots, so a mixed team would fly at least one vehicle on the wrong mass, thrust constant and inertia, which would appear in the logs as a residual and be indistinguishable from an interaction force (Section 3.3.1).

Two vehicles are the minimum for the protocol. A third is required for the three-robot scenarios, which cannot be run without it.

Each vehicle carries two additions that are not optional for this work:

A rotor-speed source , optical RPM deck or DShot telemetry. Without it the applied thrust cannot be reconstructed, so a_{res} is unavailable and *reads identically zero* rather than failing loudly. Strategies 2 and 5 additionally cannot run at all. This is the single most consequential hardware prerequisite in the setup.

A micro SD deck , the recording medium for the dataset, for the reasons in Section 5.4.

State estimation uses the on-board extended Kalman filter, fed with external pose from a motion-capture system at 100 Hz. The control loop runs at 500 Hz onboard.

5.2 Flight volume

The tracked volume measures approximately 2 m × 4 m horizontally and extends to 1.7 m in height. Two distinct limits are enforced and should not be conflated: the *tracked volume*, outside which the motion-capture system cannot see the vehicles, and an *operational floor*

of 0.30 m, chosen so that ground effect does not contaminate the measurement of inter-vehicle interaction.

The floor is lowered to 0.10 m for one scenario only, the near-ground pass of Section 5.3, whose purpose is to measure ground effect in isolation so that it can be separated from downwash rather than confounded with it.

The volume is a binding constraint rather than a formality. Several scenarios sit within approximately 10 cm of a wall at their extremes, and the scenario definitions are automatically centred and, where necessary, rotated to fit. That a configuration fits is checked before flight, not discovered during it.

The volume figures are corroborated from two independent statements but have not been tape-measured. Measure before the first session and update this section (docs/11_Hardware_Readiness_Checklist.md).

5.3 The formation library

Sixteen scenarios, fixed before any data collection and not modified afterwards. Freezing them is what allows every strategy to meet identical excitation; a library that evolved during the campaign would make later strategies incomparable with earlier ones.

The scenarios are parameterised rather than hard-coded, separations, offsets and speeds are arguments, so that a separation sweep is the same scenario at different parameters rather than a different scenario. Each is compiled to a per-vehicle polynomial trajectory and uploaded through the standard trajectory interface, so that the same command produces the same motion in simulation and on hardware.

5.3.1 The null scenarios

C1–C3, the coplanar controls in which the vehicles fly side by side at equal height, are the most important scenarios in the library and the least interesting to watch. They place the vehicles in configurations where no significant inter-vehicle interaction should exist, and they exist because of the property established in Section 3.2: the residual is a catch-all, non-zero even in isolated flight, so a residual measured in formation cannot be attributed to the interaction without a reference.

They serve two purposes. They bound the measurement floor, so that a reported improvement can be distinguished from noise. And they detect model error: a residual that appears when no neighbour is in the wash is a mis-identified mass or thrust constant, not an interaction, and must be corrected before the campaign proceeds rather than explained afterwards.

5.3.2 Regimes, and why the library spans them

The scenarios are not a list of poses but a deliberate spread across the interaction regimes that Research Question 2 distinguishes. A1, the two-robot vertical hover stack, and the coplanar controls C1–C3 are static; A2 and A3 are quasi-static; A4 and A5 sweep lateral offset; A7 sweeps separation continuously; A8 and B3 present the disturbance as a transient with a known arrival time. A model fitted on one regime and evaluated on another is what makes "outside the training distribution" a measurable condition rather than a caveat.

> **Requirement for data collection: the set must excite lateral relative motion.** A purely vertical scenario holds the lateral components of the relative state constant, so a model trained only on such data has seen no variation in

half of its six inputs and is extrapolating the first time a formation moves sideways. **The collection set must therefore include A4, A5 and A7 — not only the vertical stacks A1, A2 and A3.** This is not a preference about coverage; a set without them produces a model that cannot be evaluated on the scenarios it will meet.

This was established in simulation before any flight: a dry run collecting only a vertical stack produced training data in which two of the six network inputs never varied, which the training pipeline reports as a degenerate normalisation statistic and which no amount of training can repair.

5.4 Measurement and logging

5.4.1 Why the dataset is recorded onboard

Radio telemetry saturates at approximately two vehicles: each drone transmits on the order of 800 packets per second at full rate against a per-dongle ceiling near 1000. Recording the dataset over radio would therefore couple the sampling rate to the team size, which is unacceptable for a study whose subject is what happens as vehicles are added.

The micro SD deck records 35 variables at 500 Hz on each vehicle independently of radio and of team size. Radio logging is retained for live monitoring and for single-vehicle tuning, where the bandwidth exists.

5.4.2 What is recorded

Position and velocity estimates; attitude; gyro and accelerometer; the measured residual a_{res}; the INDI torque and filtered angular acceleration; the network prediction $\hat{a}_{\text{res}}$ and its saturation flag; the commanded setpoint; and the four motor commands.

Two of these deserve comment. The measured residual is recorded *in every controller mode*, including those that make no use of it, because the training data is collected under the uncompensated baseline. And the network prediction is recorded *whether it is enabled or not*, because comparing prediction against measurement is how a learned model is evaluated, and that comparison is only meaningful on a flight where the prediction is not also changing the vehicle's behaviour.

The logging configuration is fixed before data collection and shared by the collection and evaluation flights. A configuration change between the two would silently make the two halves of the dataset incomparable.

5.4.3 Time alignment across vehicles

Each vehicle timestamps its samples against its own boot, so the logs share no clock. Recording is started by a single broadcast, which aligns the vehicles to within the broadcast jitter plus one logging period. Residual error comes from propagation, task wake phase and crystal drift between vehicles; the last is a rate error and accumulates over a flight.

The merge tool therefore *measures* the offset by cross-correlating each vehicle's onboard trace against a radio trace sharing a common clock, and reports offset and drift rather than assuming them. The predicted alignment is a few milliseconds; that figure is to be confirmed against real logs on the first two-vehicle flight.

5.5 Protocol

5.5.1 Hardware gate

No data collected before the following sequence passes is used. The gate is not an experiment; it is the check that nothing is broken, so that what is measured afterwards means something.

1. **Validate.** Single vehicle, geometric first and then INDI, on the trajectory ladder. Confirm the gains read back as configured.
2. **Tune, then freeze.** Any remaining attitude tuning happens here and only here. The gains are then frozen for the remainder of the campaign.
3. **Two vehicles at large separation.** Confirm a_{res} is non-zero and correctly signed; confirm the measured time alignment.

The order carries the reasoning. Tuning before validating means tuning against an unknown fault. Flying two vehicles before the freeze means the gains move underneath the dataset.

Stage 1 cannot confirm that the compensation terms are *correct*, only that they are not catastrophic: with no neighbour present $a_{\text{res}} \approx 0$, so a compensation term is dormant and a clean single-vehicle flight demonstrates nothing about it. Stage 3 is the first point at which the sign of the compensation is observable at all, which is why it is part of the gate rather than the first experiment.

5.5.2 Data collection

Collection flights are flown under the *uncompensated* baseline of Section 4.2. This is not a convenience: a compensating controller partly cancels the disturbance, so the residual it records is the one that survived compensation, which is not the quantity a predictive model should be fitted to.

Separations are approached from large to small, and a tighter configuration is attempted only after the looser ones are clean.

The set must span both vertical and lateral excitation — A1/A2/A3 *and* A4/A5/A7 — for the reason given in Section 5.3. A collection campaign that omits the laterally excited scenarios cannot support the predictive strategies, and the omission is not recoverable without re-flying.

5.5.3 Comparison campaign

Each strategy flies the frozen library. Because the control law is selected at runtime (Section 4.1.3), strategies are interleaved within a session rather than flown on separate days, which pairs them against the same battery state, temperature and calibration. Each condition is repeated to allow a distribution rather than a single number to be reported.

The comparison is tightest among Strategies 1, 2, 3 and 5, which share the entire position and attitude loop and differ only in which residual term is active; a shared gain set is meaningful for them in a way it is not across structurally different controllers. Strategies 4 and 6 are compared under identical trajectories, formations and hardware, but not under an identical control structure, and the discussion chapter must phrase their results accordingly.

5.5.4 Metrics

Three axes, corresponding to the research questions.

Tracking error. Position RMSE, total and per axis, and the maximum deviation from the commanded trajectory. Alongside these, the attitude tracking error, reported as the RMSE and peak of the geometric attitude error e_R. A compensator that improves position by working the attitude loop harder has not improved tracking so much as moved the cost, and only the attitude figure makes that visible. The logging configuration described in Section 5.4.2 records attitude but not e_R itself; that signal will be added before the comparison campaign.

Residual rejection. The error in the *separation between vehicles*, measured against the commanded separation. Downwash acts as a bias, the lower vehicle sags, closing the gap, so the mean error is reported as the primary quantity and the spread as secondary. A compensator that works moves the mean back toward the commanded value. Reporting only the spread would hide precisely the effect being studied.

Alongside it, the magnitude of the measured residual that remains after compensation, and for every strategy carrying a learned model, the prediction against the measurement.

Robustness and cost. Control effort; behaviour as separation tightens; abort and failure rate; onboard computation per control step; sensing prerequisites; and the quantity of training data required to reach a given prediction accuracy.

5.6 The role of simulation

A software-in-the-loop simulator runs the same controller source that flies, with a published downwash model providing the interaction force. It was used throughout development, and its role in this thesis is deliberately limited.

Simulation is used to disprove implementations, never to validate a method. It established that every scenario in the library executes and produces the commanded geometry under both controllers, and it exposed two defects in flight code that single-vehicle testing could not have found. It cannot support a claim about how well a method works, for a reason specific to this subject: the interaction force in simulation is produced by a learned model of the same family as the one being fitted, and learning a network's output with another network is a substantially easier problem than learning real air.

No result in the two-robot results chapter or the multi-robot results chapter rests on simulation.

5.7 Threats to validity

Splitting the training and test data by contiguous flight segment rather than by sample deserves a note, because the obvious alternative is badly wrong here. At 500 Hz consecutive samples are nearly identical, so a random per-sample split places a near-duplicate of almost every test point in the training set and reports a validation error that means nothing.

Two threats are not mitigated and are stated as limitations rather than managed. The results concern a single airframe; quantities are normalised where a normalisation is defensible, but generalisation to other platforms is argued rather than shown. And battery

discharge changes the thrust constant slightly within a flight, which appears as a slow drift in the measured residual; the effect is bounded by the null scenarios but not removed.

5.8 Summary

Two or three identical brushless quadrotors, each carrying a rotor-speed source and an SD card, flying sixteen frozen scenarios inside a tracked volume of roughly $2\,\mathrm{m} \times 4\,\mathrm{m} \times 1.7\,\mathrm{m}$, with 35 variables recorded onboard at $500\,\mathrm{Hz}$ independently of team size. Gains are frozen before any data is collected; strategies are selected at runtime and interleaved within sessions; and three null scenarios establish what a measurement of zero looks like under the same pipeline.

This chapter describes a design. At the time of writing no flight has taken place. Revise to past tense as the campaign proceeds, and record deviations from this protocol here rather than only in the results.

Table 5.1: The formation library. Scenarios marked † carry elevated risk and are gated behind an explicit acknowledgement; those marked ∗ are null conditions in which no inter-vehicle interaction is expected.

	Scenario	Purpose
Two vehicles		
A1	Vertical stack, hover	Baseline. Nothing moves, so any relative displacement is interaction rather than tracking error
A2	Vertical stack, tracking	Constant separation while both vehicles move
A3	Static top	Fixed wash source, lower vehicle sweeps through: captures the transition, not only the level
A4	Offset stack	Partial overlap; asymmetric and hardest to model
A5	Reverse-circle stack	Lateral offset swept twice per lap at constant height and speed
A6	Extreme stack †	Strongest interaction, smallest error budget
A7	Dynamic merge †	One continuous sweep of the whole separation range: gives a model the gradient, not only levels
A8	Vertical swap	Wash arrives as a transient with a known arrival time; hardest case for a feedforward model
Three vehicles		
B1	I-stack	Lowest vehicle in the combined wash of two
B2	V-stack	Asymmetric aggregation, which B1 cannot produce
B3	Vertical swap	Three crossings at three height differences in one flight
Controls and coverage		
C1	Side-by-side coplanar ∗	A residual here, and not in A1, is not downwash
C2	Leader–follower line ∗	Wake rather than wash
C3	Equilateral triangle ∗	Coplanar three-body control
C4	Docking approach †	Simultaneous lateral and vertical closure
C5	Near-ground pass, one vehicle	Ground effect alone, separable from downwash

Table 5.2: Threats to validity and the design decisions taken against them.

Threat	Mitigation	Residual risk
Tuning confound	Gains frozen before collection; shared across strategies; runtime selection allows interleaving	Shared gains may suit some strategies better. Justified for 1/2/4, which share the whole loop; weaker for 3 and 6, which do not (§4.10)
Measurement floor	Null scenarios C1–C3 bound it under the same pipeline	—
Model error read as interaction	Null scenarios detect it; airframe constants identified once and frozen	Battery discharge within a flight is not compensated
Training/test leakage	Held-out scenarios; split by contiguous flight segment, never by sample	—
Session effects	Strategies interleaved within a session; repeated conditions	—
Silent loss of the residual	$a_{\text{res}} \neq 0$ verified at the start of every session	—
Simulation-to-hardware gap	No claim rests on simulation	—
Single airframe	—	**Not mitigated.** Generalisation beyond this platform is discussed, not demonstrated

Bibliography

[1] Ghulam E Mustafa Abro, J. Alsayegh, and Hifza Mustafa. High-precision uav swarm control: Evaluating fractional-order methods for close-proximity operations. *IEEE Journal of Emerging and Selected Topics in Industrial Electronics*, 2025. Abstract read via OpenAlex; full text not held (paywalled).

[2] Leonard Bauersfeld, Koen Muller, Dominic Ziegler, Filippo Coletti, and Davide Scaramuzza. Robotics meets fluid dynamics: A characterization of the induced airflow below a quadrotor as a turbulent jet. arXiv:2403.13321, 2024. Published in IEEE Robotics and Automation Letters.

[3] Kong Yao Chee, Pei-An Hsieh, George J. Pappas, and M. Ani Hsieh. Flying quadrotors in tight formations using learning-based model predictive control. arXiv:2410.09727, 2024.

[4] Eckart Cobo-Briesewitz, Khaled Wahba, and Wolfgang Hönig. Learned incremental nonlinear dynamic inversion for quadrotors with and without slung payloads. arXiv:2503.09441, 2025. Published in Proceedings of the 8th Annual Conference on Learning for Dynamics and Control (L4DC), PMLR.

[5] Jennifer Gielis, Ajay Shankar, Ryan Kortvelesy, and Amanda Prorok. Modeling aggregate downwash forces for dense multirotor flight. arXiv:2312.03488, 2023.

[6] Weibin Gu, Jiance Zhao, and Alessandro Rizzo. Learning uncertainties online for quadrotor flight control: A comparative study. *Journal of Intelligent & Robotic Systems*, 2025. Abstract read via OpenAlex; full text not held (paywalled).

[7] Pei-An Hsieh, Kong Yao Chee, and M. Ani Hsieh. Online adaptation for flying quadrotors in tight formations. arXiv:2506.17488, 2025.

[8] Pei-An Hsieh, Fengjun Yang, Nikolai Matni, and M. Ani Hsieh. Flatness-preserving residual learning for real-time tight quadrotor formation flight. arXiv:2607.12275, 2026.

[9] Anoop Kiran, Nora Ayanian, and Kenneth Breuer. Influence of static and dynamic downwash interactions on multi-quadrotor systems. arXiv:2507.09463, 2025.

[10] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Control of complex maneuvers for a quadrotor uav using geometric methods on se(3). arXiv:1003.2005, 2010. COMPANION arXiv paper. Used for equation-level verification of the objects shared with CDC 2010: the attitude error function, the attitude and angular velocity errors, and the full torque law with feedforward. Cite CDC 2010 as primary for Strategy 0. The extended v4 revision, and only it, carries the three flight modes, velocity mode, hybrid concatenation, the heading-projection proposition and the switched

720/360-degree example; cite THIS key, not CDC, for any of those. Do not copy v4 region-of-attraction bounds or Lyapunov matrices onto the CDC citation.

[11] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). In *49th IEEE Conference on Decision and Control (CDC)*, pages 5420–5425, 2010. PRIMARY reference for the geometric SE(3) controller (Strategy 0). Tracks position and one heading direction, four outputs. Its torque law is CDC eq. (16) and includes the angular feedforward term; the firmware implements a reduced form of it. Stability is regional (Prop. 2 exponential below 90 degrees; Prop. 3 almost-global attractiveness below 180 degrees), not global. Its companion is arXiv:1003.2005v1. Do NOT attribute the three flight modes, velocity mode, heading-projection proposition or the switched 720/360-degree example to this paper; those are in the v4 companion only.

[12] Jinjie Li, Liang Han, Haoyang Yu, Yuheng Lin, Qingdong Li, and Zhang Ren. Nonlinear mpc for quadrotors in close-proximity flight with neural network downwash prediction. arXiv:2304.07794, 2023.

[13] Ajay Shankar, Heedo Woo, and Amanda Prorok. Docking multirotors in close proximity using learnt downwash models. arXiv:2311.13988, 2023.

[14] Guanya Shi, Wolfgang Hönig, Xichen Shi, Yisong Yue, and Soon-Jo Chung. Neural-swarm2: Planning and control of heterogeneous multirotor swarms using learned interactions. arXiv:2012.05457, 2020. Published in IEEE Transactions on Robotics.

[15] Guanya Shi, Wolfgang Hönig, Yisong Yue, and Soon-Jo Chung. Neural-swarm: Decentralized close-proximity multirotor control using learned interactions. arXiv:2003.02992, 2020.

[16] Ewoud J. J. Smeur, Guido C. H. E. de Croon, and Qiping Chu. Cascaded incremental nonlinear dynamic inversion control for mav disturbance rejection. arXiv:1701.07254, 2017.

[17] H. Smith, A. Shankar, J. Gielis, J. Blumenkamp, and A. Prorok. So(2)-equivariant downwash models for close proximity flight. arXiv:2305.18983, 2023.

[18] Ezra Tal and Sertac Karaman. Accurate tracking of aggressive quadrotor trajectories using incremental nonlinear dynamic inversion and differential flatness. arXiv:1809.04048, 2018.

[19] Fengjun Yang, Jake Welde, and Nikolai Matni. Learning flatness-preserving residuals for pure-feedback systems. arXiv:2504.04324, 2025.

[20] Ruiqi Zhang, Dingqi Zhang, and Mark W. Mueller. Proxfly: Robust control for close proximity quadcopter flight via residual reinforcement learning. arXiv:2409.13193, 2024.